

Matricule N°: _____ - _____ - _____.

Nom : _____.

Prénom : _____.

Programme : _____.

GIF-1003– Programmation avancée en C++

Examen final Automne 2020

Durée: 2h50

Aucun document autorisé

Prénom	Points	Pages										
		1	2	3	4	5	6	7	8	9	10	11
Idrissa	33						8	6	16		3	
Ndiogou	25									13		12
Yao	30	6	6		11	7						
Diego	12			12								

ail compte.

contenir vos

p1	6
p2	6
p3	12
p4	11
p5	7
p6	8
p7	6
p8	16
p9	13
p10	3
p11	12
Total	

Pour la correction

1. Principes (12 points)

Théorie du contrat (4 points)

Jean utilise une classe développée par Rémi dans son projet. Situez les responsabilités si Jean compile son programme et trouve les erreurs suivantes:

1. PRECONDITIONEXCEPTION (☒ Jean ☐ Rémi).
2. POSTCONDITIONEXCEPTION (☐ Jean ☒ Rémi).
3. INVARIANTEXCEPTION (☐ Jean ☒ Rémi).

(2)

- 1/ réponse fautive

À quoi correspond l'invariant d'une classe?

a) Des conditions qui précisent ce qui ne peut pas varier dans une classe	b) Des conditions qui précisent les contraintes de validité à respecter au moment de la construction des objets par la classe	c) Des conditions qui précisent les contraintes de validité des objets devant être respectées pendant toute leur durée de vie de leur construction à leur destruction	d) Des contraintes de validité qui doivent être vérifiées dans toutes les méthodes de la classe
---	---	---	---

(Indiquer la ou les bonnes réponses) c

(2)

Gestion mémoire (4 points)

Dans quels cas de figure le constructeur copie est-il appelé implicitement ?

- passage par valeur
 - retour par valeur

(2)

De ce qui suit, qu'est-ce qui est automatiquement ajouté à toute classe si on ne l'implémente pas

a) le constructeur de copie	b) l'opérateur d'assignation	c) un constructeur sans paramètre
-----------------------------	------------------------------	-----------------------------------

(Indiquer la ou les bonnes réponses) a b c (2)

Polymorphisme (4 points)

Par quels moyens et comment le liage dynamique (polymorphisme) est-il réalisé ?

vtable

- objet dérivé dans un pointeur de la classe de base (1)

@ de la vtable dans les objets

- vtable -> contient les adresses des méthodes à appeler sur l'objet courant (par déréférencement du pointeur) (2)

déclarée virtuelle (1)

2. Application

Nous voulons développer un programme afin de gérer l'inventaire d'une épicerie. Ce programme est construit au fur et à mesure. Sa construction est guidée par une série de questions indépendantes.

Dans tout le développement, la théorie du contrat doit être appliquée (il ne sera pas toujours précisé explicitement dans les questions qui suivent qu'il faut le faire. Alors, pensez-y systématiquement.

Pour cela vous disposez des macros définies dans le module `ContratException` (fichiers `ContratException.h` et `ContratException.cpp`):

`PRECONDITION(test logique)`

`INVARIANT(test logique)`

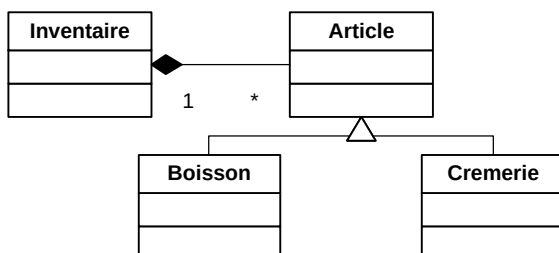
`POSTCONDITION(test logique)`

`INVARIANTS()`

`INVARIANTS()` appelle la méthode `verifieInvariant` que vous allez devoir implémenter.

Hierarchie de classe, Héritage et polymorphisme

Les données manipulées correspondent à des articles de différentes catégories. Voici la hiérarchie de classe utilisée :



ces classes seront implémentées dans l'espace de nom

examenFinal

Toutes les interfaces de ces classes sont données, mais ne sont pas tout à fait complètes. **Il faut les compléter au fur et à mesure si besoin.** Certaines méthodes sont déclarées dans ces interfaces, mais ne sont pas à implémenter, supposant qu'elles le sont déjà. Pensez à les utiliser au besoin.

Interface de la classe Article (6 points):

// Fichier : Article.h

#ifndef ARTICLE_H_DEJA_INCLU

#define ARTICLE_H_DEJA_INCLU

#include <string>

6

class Article

{

~Article () {};

Article* clone () ;

std::string reqArticleFormate () ;

reqReference () ;//accesseur

reqQuantite () ;//accesseur

std::string m_reference;

int m_quantite;

};

#endif

Interface de la classe Boisson (6 points):

// Fichier : Boisson.h

#ifndef BOISSON_H_DEJA_INCLU

#define BOISSON_H_DEJA_INCLU

class Boisson

{

Article* clone () ;

std::string reqArticleFormate () ;

float reqContenance () ;//accesseur

float m_contenance;

};

#endif

6

```

/**
 * \file Article.h
 */
#ifndef ARTICLE_H_DEJA_INCLU
#define ARTICLE_H_DEJA_INCLU

#include "ContratException.h"
#include <string>
namespace examenFinal
{
    class Article
    {
    public:
        Article(const std::string& p_reference, unsigned p_quantite = 0);
        virtual ~Article() {};
        virtual Article* clone() const = 0;
        virtual std::string reqArticleFormate() const = 0;
        virtual unsigned reqQuantite() const;
        const std::string& reqReference() const;

    private:
        void verifieInvariant() const;
        std::string m_reference; //code de l'article
        unsigned m_quantite; // Quantité en stock
    };
}

//*****
// Fichier : Boisson.h
//*****
#ifndef BOISSON_H_DEJA_INCLU
#define BOISSON_H_DEJA_INCLU

#include "ContratException.h"
#include "Article.h"
namespace examenFinal
{
    class Boisson: public Article
    {
    public:
        Boisson(const std::string& uneReference, intunsigned quantite, float contenance);
        virtual Article* clone() const;
        virtual std::string reqArticleFormate() const;
        float reqContenance() const;

    private:
        void verifieInvariant() const;
        float m_contenance;
    };
} //namespace examenFinal
#endif

```

Handwritten annotations:

- ① (circled) next to `namespace examenFinal`
- ① (circled) next to `Article` class definition
- ① (circled) next to `Article` constructor
- ① (circled) next to `virtual Article* clone() const = 0;`
- ① (circled) next to `virtual std::string reqArticleFormate() const = 0;`
- ① (circled) next to `virtual unsigned reqQuantite() const;`
- ① (circled) next to `const std::string& reqReference() const;`
- ① (circled) next to `private:`
- ① (circled) next to `void verifieInvariant() const;`
- ① (circled) next to `std::string m_reference;`
- ① (circled) next to `unsigned m_quantite;`
- ① (circled) next to `Boisson` class definition
- ① (circled) next to `Boisson` constructor
- ① (circled) next to `virtual Article* clone() const;`
- ① (circled) next to `virtual std::string reqArticleFormate() const;`
- ① (circled) next to `float reqContenance() const;`
- ① (circled) next to `private:`
- ① (circled) next to `void verifieInvariant() const;`
- ① (circled) next to `float m_contenance;`

Other handwritten notes:

- declaration virtuelle pure* (with an arrow pointing to the `clone` and `reqArticleFormate` methods)
- (-0.5)* (with an arrow pointing to the `reqReference` method)
- int* (with an arrow pointing to the `quantite` parameter in the `Boisson` constructor)

Interface de la classe Inventaire (11 points):

```
// Fichier Inventaire.h
```

```
#ifndef INVENTAIRE_H_DEJA_INCLU
```

```
#define INVENTAIRE_H_DEJA_INCLU
```

```
#include <vector>
```

```
#include "ContratException.h"
```

```
#include "Date.h"
```

```
class Inventaire
```

```
{
```

```
    Inventaire(const util::Date& uneDate);
```

```
    reqDate      ()          ;//accesseur
```

```
    std::string  reqInventaireFormate ()          ;
```

```
    util::Date   m_date; // date de l'inventaire
```

```
};
```

```
#endif
```

Inventaire.h

```

1/**
2 * \file Inventaire.h
3 * \brief Classe : Inventaire
4 * Attributs : Date m_date La date de l'inventaire >01 janvier 2000
5 * \author theud1
6 * \version 0.1
7 * \date 2014-11-27
8 */
9#ifndef INVENTAIRE_H_DEJA_INCLU
10#define INVENTAIRE_H_DEJA_INCLU
11
12#include <vector>
13#include "ContratException.h"
14#include "ArticleException.h"
15#include "Date.h"
16#include "Article.h"
17namespace examenFinal
18{
19class Inventaire
20{
21public:
22    Inventaire(const util::Date& uneDate);
23    ~Inventaire();
24    Inventaire(const Inventaire& p_inventaire);
25    void ajouterArticle(const Article& nouvelArticle);
26
27    void asgDate(long jour, long mois, long annee);
28    std::string reqInventaireFormate() const;
29    const util::Date& reqDate() const;
30
31private:
32    util::Date m_date; // date de l'inventaire
33    std::vector<Article*> m_vArticles;
34    bool ArticleEstDejaPresent(const std::string& p_reference) const;
35    void verifieInvariant() const;
36};
37
38} // namespace examenFinal
39#endif
40

```

Handwritten annotations on the code:

- Line 14: circled 1
- Line 15: circled 1
- Line 16: circled 1
- Line 17: circled 1
- Line 22: circled 1
- Line 23: circled 1
- Line 24: circled 1
- Line 25: circled 1
- Line 26: circled 1
- Line 27: circled 1
- Line 28: circled 1
- Line 29: circled 1
- Line 30: circled 1
- Line 31: circled 1
- Line 32: circled 1
- Line 33: circled 1
- Line 34: circled 1
- Line 35: circled 1
- Line 36: circled 1
- Line 37: circled 1
- Line 38: circled 1
- Line 39: circled 1
- Line 40: circled 1

Handwritten notes:

- Line 25: ajouterArticle
- Line 28: reqInventaireFormate
- Line 29: reqDate
- Line 34: ArticleEstDejaPresent
- Line 35: verifieInvariant

Pour la classe `Article` (9 points)

On vous demande d'écrire les fonctions membres suivantes :

- un constructeur avec paramètres. Les paramètres sont la référence qui doit commencer par 3 lettres et être composée d'au moins 8 caractères et la quantité (pensez au contrat). Prévoir ici l'en-tête de commentaires de spécification qui seront utilisés pour générer la documentation avec Doxygen (voir l'annexe pour les balises). Cet en-tête doit être complet.

7

Les commentaires de spécification ne sont pas demandés dans ce qui suit.

- La fonction `reqArticleFormate` qui doit être implantée **systematiquement** dans toute classe dérivée de la classe `Article` pour permettre le polymorphisme. Pensez à faire une déclaration en conséquence. Cette méthode retourne un objet string avec mise en forme ayant le format suivant :

liqueur256 50

Pour un article de référence `liqueur256` en quantité de 50 (pensez à utiliser une `ostream`. La méthode `str` appliquée sur une `ostream` retourne un objet string)

Il n'est pas demandé de la définir ici, supposant qu'elle l'est déjà.

Article.cpp

```

1//*****
2// Fichier : Article.cpp
3// Classe : Article
4//*****
5#include "Article.h"
6#include <string>
7#include <sstream>
8
9using namespace std;
10namespace examenFinal
11{
12
13/**
14 * \brief Constructeur avec paramètres
15 * \param[in] p_reference la référence de l'article
16 * \pre commence par 3 lettres
17 * \pre est composé d'au moins 8 caractères
18 * \param[in] p_quantite
19 */
20Article::Article(const std::string& p_reference, unsigned p_quantite) :
21    m_reference(p_reference), m_quantite(p_quantite)
22{
23    PRECONDITION(isalpha(p_reference[0]) && isalpha(p_reference[1]) &&
24        isalpha(p_reference[2]));
25    PRECONDITION(p_reference.size() >= 8);
26    // m_reference = p_reference;
27    // m_quantite = p_quantite;
28    POSTCONDITION(m_reference == p_reference);
29    POSTCONDITION(m_quantite == p_quantite);
30    INVARIANTS();
31}
32std::string Article::reqArticleFormate() const
33{
34    ostringstream os_format;
35    os_format << "Ref: " << m_reference << endl << "Quantite : " << m_quantite
36    << endl;
37    return (os_format.str());
38}
39
40void Article::verifieInvariant() const
41{
42    INVARIANT(isalpha(m_reference[0]) && isalpha(m_reference[1]) &&
43        isalpha(m_reference[2]));
44    INVARIANT(m_reference.size() >= 8);
45}
46unsigned Article::reqQuantite() const
47{
48    return m_quantite;
49}
50
51const std::string& Article::reqReference() const

```


La fonction `verifieInvariant` pour pouvoir implanter le contrat

2

Complétez `Article.h` (pensez aussi à compléter les déclarations des deux accesseurs, vous n'avez cependant pas à écrire leur implémentation)

La classe `Article` est une classe abstraite (voir plus loin).

Pour la classe `Boisson` (12 points)

La classe `Boisson` est dérivée de la classe `Article` tel que présenté plus haut. On vous demande d'écrire les fonctions membres suivantes :

- un constructeur avec paramètres. Les paramètres sont une référence, la quantité en stock et la contenance en litres qui doit être positive, mais inférieure à 5.

5

La fonction `verifieInvariant` pour pouvoir implanter le contrat

1

Boisson.cpp

```

1//*****
2//
3// Fichier : Boisson.cpp
4// Classe : Boisson
5//
6//*****
7// 13 avril 2005 Thierry EUDE version initiale
8// révisé: 2012-04-07
9//*****
10#include <sstream>
11#include "Boisson.h"
12
13using namespace std;
14namespace examenFinal
15{
16    Boisson::Boisson(const string& uneReference, unsigned quantite, float contenance) :
17        Article(uneReference, quantite), m_contenance(contenance)
18    {
19        PRECONDITION(contenance>0 && contenance<5); ①
20        // m_contenance = contenance; ← ③
21        POSTCONDITION(m_contenance == contenance); ①
22        INVARIANTS();
23    }
24
25    Article* Boisson::clone() const
26    {
27        return new Boisson(*this); ②
28    }
29
30    std::string Boisson::reqArticleFormate() const
31    {
32        ostringstream os_Boisson;
33        os_Boisson << "Boisson" << endl << Article::reqArticleFormate() ③ ②
34            << "Contenance :" << m_contenance << endl;
35        return os_Boisson.str(); ①
36    }
37
38    void Boisson::verifieInvariant() const
39    {
40        INVARIANT (m_contenance>0 && m_contenance<5); ①
41    }
42
43    float Boisson::reqContenance() const
44    {
45        return m_contenance;
46    }
47} //namespace examenFinal
48

```

- La méthode `clone` qui doit être implantée **systematiquement** dans toute classe dérivée de la classe `Article`, permet de faire une copie sur le monceau (tas ou heap) de l'objet courant. Ceci permet d'ajouter des `Articles` dans l'inventaire tout en autorisant le polymorphisme (dans la classe `Inventaire`)
-
-
-
-
-

- La méthode `reqArticleFormate` qui retourne un objet string avec mise en forme ayant le format suivant :

Boisson liqueur256 50 1

Pour un article `Boisson` (nom de la classe), de référence `liqueur256`, en quantité de 50 et de contenance 1litre
(pensez à utiliser une `ostream`. La méthode `str` appliquée sur une `ostream` retourne un objet string)

Pour la classe `Inventaire` (29 points):

La classe `Inventaire` comprend deux attributs; la date de l'inventaire et un vector contenant des pointeurs vers des `Articles` (qui peuvent être des objets `Boisson`, `Cremerie`, ...) de l'inventaire. On vous demande d'écrire les fonctions membres suivantes :

- une méthode `ajouterArticle`, qui permet d'ajouter des `Articles` à l'inventaire. La méthode `push_back` appliquée sur un `vector` permet d'ajouter un élément passé en paramètre à la fin du `vector` seulement si cet article n'est pas déjà présent dans celui-ci. La méthode "interne à la classe" `articleEstDejaPresent` permet de le vérifier (à implémenter ci-dessous). Dans le cas où l'inventaire contient déjà cet article, une exception du type `ArticleDejaPresentException` sera lancée. La classe `Inventaire` est responsable de la gestion de la mémoire.

ajouterArticle (

- La méthode `articleEstDejaPresent`
`articleEstDejaPresent (const std::string& p_reference)`

- un destructeur.

Inventaire.cpp

```

1/**
2 * \file Inventaire.cpp
3 * \date 2014-11-27
4 */
5#include <sstream>
6#include "Inventaire.h"
7#include "Date.h"
8#include "ArticleException.h"
9
10using namespace std;
11using namespace util;
12
13namespace examenFinal
14{
15    Inventaire::Inventaire(const Date& uneDate) : ) (1)
16    {
17        m_date(uneDate)
18        PRECONDITION (Date (01,01,2000) < uneDate); (2)
19        // m_date.asgDate(uneDate.reqJour(),uneDate.reqMois(),uneDate.reqAnnee());
20        POSTCONDITION(m_date == uneDate);
21        INVARIANTS();
22    }
23
24    Inventaire::~Inventaire()
25    { //solution avec iterator
26        vector<Article*>::iterator it_reference;
27        for (it_reference = m_vArticles.begin(); it_reference < m_vArticles.end(); it_reference++)
28        {
29            delete *it_reference; (3)
30        }
31        //solution avec indice
32        for (unsigned i = 0; i < m_vArticles.size(); i++) ) (2)
33        {
34            delete m_vArticles[i]; (3)
35        }
36    }
37
38    void Inventaire::asgDate(long jour, long mois, long annee)
39    {
40        Date uneDate(jour, mois, annee);
41        PRECONDITION(Date(1,1,2000)<uneDate);
42        m_date.asgDate(jour, mois, annee);
43        POSTCONDITION(m_date == uneDate);
44        INVARIANTS();
45    }
46
47    void Inventaire::ajouterArticle(const Article& nouvelArticle)
48    {
49        if (ArticleEstDejaPresent(nouvelArticle.reqReference())) (2)
50        {
51            throw ArticleDejaPresentException("\n Cet article existe deja !"); (2)
52        }
53        else
54        {
55            m_vArticles.push_back(nouvelArticle.clone()); (2)
56        }
57    }

```

Inventaire.cpp

```

58
59 std::string Inventaire::reqInventaireFormate() const
60 {
61     ostringstream inventaire;
62     vector<Article*>::const_iterator iterI = m_vArticles.begin(); ①
63
64     inventaire << "Inventaire du : " << m_date.reqDateFormatee() << endl
65     << "-----" << endl << m_vArticles.size() ①
66     << " article(s) enregistre(s)" << endl;
67     while (iterI != m_vArticles.end()) ①
68     {
69         inventaire << (*iterI)->reqArticleFormate() << endl; ②
70         iterI++; ①
71     }
72     return inventaire.str();
73 }
74
75 bool Inventaire::ArticleEstDejaPresent(const std::string & p_reference) const
76 {
77     bool existe = false;
78     //solution avec iterator
79     vector<Article *>::const_iterator it_reference;
80     for (it_reference = m_vArticles.begin(); it_reference < m_vArticles.end(); it_reference++)
81     {
82         if ((*it_reference)->reqReference() == p_reference) ② ③
83             existe = true;
84     }
85     //solution avec indice
86     unsigned i;
87     for (i = 0; i < m_vArticles.size(); i++) ②
88     {
89         if (m_vArticles[i]->reqReference() == p_reference) ③
90             existe = true;
91     }
92     return existe;
93 }
94
95 void Inventaire::verifieInvariant() const
96 {
97     INVARIANT (Date (01,01,2000) < m_date);
98 }
99
100 const util::Date& Inventaire::reqDate() const
101 {
102     return m_date;
103 }
104 //namespace examenFinal
105
106
107

```

en cohérence avec le h

- une méthode `reqInventaireFormate` qui retourne un objet string formaté (voir le prototype dans l'interface). **Utiliser ici un iterator.** Le format de l'objet string doit être le suivant :

Inventaire du : Dimanche le 26 novembre 2017 ----- 1 article(s) enregistre(s) Boisson Ref: liqueur256 Quantite : 50 Contenance :1	Dans l'exemple, l'inventaire du 26/11/2017 compte 1 article de catégorie Boisson, mais il pourrait y en avoir plus d'un.
---	--

(Pensez à utiliser une `ostream`, la méthode `reqDateFormatee` appliquée sur un objet `Date` retourne celui-ci sous la forme d'un objet string formaté comme suit : `Dimanche le 26 novembre 2017`)

- Un constructeur de copie

```

Inventaire::Inventaire(const Inventaire& p_inventaire) :
    m_date(p_inventaire.m_date)
{
    for (unsigned i = 0; i < m_vArticles.size(); ++i)
    {
        ajouterArticle(*p_inventaire.m_vArticles[i]);
    }
}

```

Diagramme de copie de code avec annotations :

- Le code est copié dans un éditeur de texte.
- Le code est coloré : `const` est en vert, `Inventaire` est en bleu, `m_date` est en bleu, `for` est en rouge, `unsigned` est en rouge, `i` est en bleu, `0` est en bleu, `<` est en bleu, `m_vArticles.size()` est en bleu, `++i` est en bleu, `*` est en bleu, `p_inventaire.m_vArticles[i]` est en bleu, `ajouterArticle` est en bleu.
- Des annotations rouges sont présentes :
 - Un "2" rouge est placé sous `m_date(p_inventaire.m_date)`.
 - Un "2" rouge est placé sous la fermeture de la boucle `)`.
 - Un "3" rouge est placé sous `ajouterArticle(*p_inventaire.m_vArticles[i]);`.

Rappel : Avez-vous pensé à mettre à jour les interfaces ?

3. Classe ArticleDejaPresentException (3 points)

Cette classe permet de gérer l'exception de l'ajout d'un doublon d'article dans l'inventaire. Cette classe hérite de `std::runtime_error` parce que cette exception est une erreur qui pourra toujours se produire, ce n'est pas une erreur de programmation.

Elle contient seulement une méthode qui est le constructeur suivant :

```
ArticleDejaPresentException (const std::string& p_raison);
```

Ce constructeur ne fait qu'appeler le constructeur de la classe parent en lui passant la raison comme paramètre. La méthode `what()` de la classe `std::exception` au haut de la hiérarchie permet d'obtenir la string qu'on a passé.

On vous demande de compléter l'interface de cette classe

```
// Fichier ArticleDejaPresentException.h
```

```
#include <stdexcept>
```

```
class ArticleDejaPresentException
```

```
{
```

et d'écrire l'implémentation du constructeur.

```
ArticleDejaPresentException (const std::string& p_raison)
```

4. Test unitaire (12 points)

On vous demande d'implémenter le test unitaire du constructeur de la classe Article, ceci en utilisant les fonctionnalités offertes par le framework GoogleTest.

Les principales macros et assertions sont rappelées en annexe.

```
/**
 * \file ArticleTesteur.cpp
 * Implantation des tests unitaires de la classe Article
 */
#include<gtest/gtest.h>
#include "Article.h"

using namespace std;

using namespace examenFinal;
```



```
/*  
 * ArticleException.h  
 */  
#ifndef ARTICLEEXCEPTION_H_  
#define ARTICLEEXCEPTION_H_  
#include <stdexcept>  
#include <string>  
namespace examenFinal  
{  
class ArticleDejaPresentException: public std::runtime_error  
{  
public:  
    ArticleDejaPresentException(const std::string& raison);  
};  
#endif /* ARTICLEEXCEPTION_H_ */
```

```
*  
 * ArticleException.cpp  
 *  
 * Created on: 2012-04-07  
 * Author: theud1  
 */  
  
#include "ArticleException.h"
```

```
namespace examenFinal  
{  
ArticleDejaPresentException::ArticleDejaPresentException(  
    const std::string& raison) :  
    std::runtime_error(raison)  
{  
}  
}
```

ArticleTesteur.cpp

```
1/**
2 * \file ArticleTesteur.cpp
3 * Implantation des tests unitaires de la classe Article
4 * Article étant une classe abstraite, on crée une classe concrète
5 * dérivée de cette classe afin de pouvoir effectuer les tests
6 * \date 2012-04-07
7 */
8#include<gtest/gtest.h>
9#include "Article.h"
10#include<fstream>
11using namespace std;
12using namespace examenFinal;
13
14// Création de la classe concrète de Article pour les tests
15class ArticleDeTest: public Article
16{
17public:
18    ArticleDeTest(const std::string& uneReference, int quantite) :
19        Article(uneReference, quantite)
20    {};
21    virtual ~ArticleDeTest(){};
22    virtual string reqArticleFormate() const
23    {return Article::reqArticleFormate();};
24    virtual Article* clone() const {return new ArticleDeTest(*this);};
25};
26//*****
27// Test du Constructeur
28// Cas valide: ConstructeurParametresValides : Création d'un objet Article et
29//              vérification de l'assignation de tous les attributs
30// Cas invalides:
31// ConstructeurReferenceLettresInvalide: Création avec référence ne commençant par
32// 3 lettres
33// ConstructeurReferenceCaractereInvalide: Création avec référence comportant
34// moins de 8 caractères
35//*****
36TEST(UneArticleConstructeur, ConstructeurParametresValides)
37{
38    ArticleDeTest unArticle("liqueur256", 50);
39    ASSERT_EQ("liqueur256", unArticle.reqReference());
40    ASSERT_EQ((unsigned)50, unArticle.reqQuantite());
41}
42// Cas invalides:
43// ConstructeurReferenceLettresInvalide: Création avec référence ne commençant par
44// 3 lettres
45TEST(UneArticleConstructeur, ConstructeurReferenceLettresInvalide)
46{
47    ASSERT_THROW(ArticleDeTest("t1estdevalidite", 235), PreconditionException);
48}
49// ConstructeurReferenceCaractereInvalide: Création avec référence comportant
50// moins de 8 caractères
51TEST(UneArticleConstructeur, ConstructeurReferenceCaractereInvalide)
52{
53    ASSERT_THROW(ArticleDeTest("test", 235), PreconditionException);
54}
55}
```

[illegible]

Annexe

```
size_t size() const;
```

Return length of string

Returns the length of the string, in terms of number of characters.

This is the number of actual characters that conform the contents of the [string](#), which is not necessarily equal to its storage [capacity](#).

Both `string::size` and [string::length](#) are synonyms and return the same value.

Parameters

None

Return Value

The number of characters in the string.

[size_t](#) is an unsigned integral type.

```
int isalnum ( char c );
```

Checks whether *c* is either a decimal digit or an uppercase or lowercase letter.

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is either a digit or a letter. Zero (i.e., `false`) otherwise.

```
int isalpha ( char c );
```

Checks whether *c* is an alphabetic letter.

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is an alphabetic letter. Zero (i.e., `false`) otherwise.

```
int isdigit ( char c );
```

Checks whether *c* is a decimal digit character.

Decimal digits are any of: 0 1 2 3 4 5 6 7 8 9

Parameters

c

Character to be checked

Return Value

A value different from zero (i.e., `true`) if indeed *c* is a decimal digit. Zero (i.e., `false`) otherwise.

`ostringstream` fournit une interface pour manipuler des chaînes de caractères comme des flux de sortie.

membres public :

```
string str ( ) const; pour obtenir l'objet string associé (fonction membre publique)
```

La méthode `size()` appliquée sur un vector retourne le nombre d'éléments qu'il contient.

La méthode `begin()` appliquée sur un vector retourne un iterator pointant sur le début de celui-ci.

La méthode `end()` appliquée sur un vector retourne un iterator pointant un élément passé la fin de celui-ci.

Balises Doxygen

///
... Documentation ...

\file

Permet de créer un bloc de documentation pour un fichier source ou d'en-tête.

\brief

Permet de créer une courte description dans un bloc de documentation. La description peut se faire sur une seule ligne ou plusieurs.

\author

Permet d'ajouter un nom d'auteur (ex: l'auteur d'un fichier ou d'une fonction).

\version

Permet l'ajout du numéro de version (ex: dans le bloc de documentation d'un fichier).

\date

Permet l'ajout d'une date.

\struct

Permet la création d'un bloc de documentation pour une structure.

\enum

Permet la création d'un bloc de documentation pour une énumération de constantes.

\fn

Permet la création d'un bloc de documentation pour une fonction ou méthode.

\def

Permet la création d'un bloc de documentation pour une macro ou constante symbolique.

\param

Permet d'ajouter un paramètre dans un bloc de documentation d'une fonction ou d'une macro. Cette balise permet aussi d'ajouter en option un tag pour montrer qu'il s'agit d'un argument entrant, sortant ou les deux en précisant au choix: **[in]**, **[out]**

\return

Permet de décrire le retour d'une fonction ou d'une méthode.

\bug

Permet de commencer un paragraphe décrivant une bogue (ou plusieurs).

\class

Permet de créer un bloc de documentation d'une classe (C++). L'utilisation est identique à la balise \struct

\namespace

Permet de créer un bloc de documentation pour un espace de nom (C++).

Framework GoogleTest :

Assertions Basiques

Fatal assertion	Nonfatal assertion	Vérifie
ASSERT_TRUE(<i>condition</i>);	EXPECT_TRUE(<i>condition</i>);	<i>condition</i> is true
ASSERT_FALSE(<i>condition</i>);	EXPECT_FALSE(<i>condition</i>);	<i>condition</i> is false

Comparaisons binaires (comparaisons entre deux valeurs v1 et v2 (valeurs comparables))

Fatal assertion	Nonfatal assertion	Vérifie
ASSERT_EQ(<i>expected</i> , <i>actual</i>);	EXPECT_EQ(<i>expected</i> , <i>actual</i>);	<i>expected</i> == <i>actual</i>
ASSERT_NE(<i>val1</i> , <i>val2</i>);	EXPECT_NE(<i>val1</i> , <i>val2</i>);	<i>val1</i> != <i>val2</i>
ASSERT_LT(<i>val1</i> , <i>val2</i>);	EXPECT_LT(<i>val1</i> , <i>val2</i>);	<i>val1</i> < <i>val2</i>
ASSERT_LE(<i>val1</i> , <i>val2</i>);	EXPECT_LE(<i>val1</i> , <i>val2</i>);	<i>val1</i> <= <i>val2</i>
ASSERT_GT(<i>val1</i> , <i>val2</i>);	EXPECT_GT(<i>val1</i> , <i>val2</i>);	<i>val1</i> > <i>val2</i>
ASSERT_GE(<i>val1</i> , <i>val2</i>);	EXPECT_GE(<i>val1</i> , <i>val2</i>);	<i>val1</i> >= <i>val2</i>

Assertions d'exceptions

Fatal assertion	Nonfatal assertion	Vérifie
<code>ASSERT_THROW(statement, exception_type);</code>	<code>EXPECT_THROW(statement, exception_type);</code>	<i>statement</i> throws an exception of the given type
<code>ASSERT_ANY_THROW(statement);</code>	<code>EXPECT_ANY_THROW(statement);</code>	<i>statement</i> throws an exception of any type
<code>ASSERT_NO_THROW(statement);</code>	<code>EXPECT_NO_THROW(statement);</code>	<i>statement</i> doesn't throw any exception

Test simple:

```
TEST(nom_cas_test, nom_test)
{
    ... corps de test ...
}
```

Tests utilisant un fixture, utiliser TEST_F()

Structure d'un fixture :

```
class FooTest : public ::testing::Test {
//constructeur
...
//méthodes
...
//attributs utilisés dans les tests, initialisés dans le constructeur
...
};
```